

NetworkDevicesMonitor

Ivaniciuc Teodor-Arsenie

Universitatea "Alexandru Ioan Cuza" din Iasi, Facultatea de Informatica
Iasi, Romania

`teodor-arsenie.ivaniciuc@info.uaic.ro`

Abstract. NetworkDevicesMonitor este o platforma software completa pentru monitorizarea si analiza infrastructurii IT, integrand protocoale standardizate (Syslog) cu agenti personalizati pentru colectarea telemetriei. Arhitectura multi-threaded implementeaza pattern-uri avansate (Producer-Consumer, Thread-Safe Queues) si ofera o interfata grafica asincrona Qt6 cu dashboard-uri interactive, vizualizari in timp real, sistem de alertare automata si filtrare avansata. Proiectul demonstreaza aplicarea conceptelor de retele, concurenta, baze de date si design UI/UX profesional, incluzand functionalitati complete de securitate prin whitelist/blacklist, management alerte personalizate si filtre dinamice.

1 Introducere

1.1 Viziunea Generala și Motivatie

in infrastructurile IT moderne, echipamentele de retea (routere, switch-uri, firewall-uri) si statiile de lucru genereaza volume masive de date sub forma de log-uri si metrici. Monitorizarea manuala devine rapid impracticabila. Lipsa unui sistem centralizat conduce inevitabil la:

- Pierderea evenimentelor critice de securitate.
- Dificultati majore in diagnosticarea problemelor de performanta.
- Lipsa vizibilitatii centralizate asupra infrastructurii.
- Timp de raspuns crescut la incidente.
- Absenta filtrarii inteligente a datelor relevante (zgomot informational).
- Management ineficient al alertelor si notificarilor.

1.2 Obiectivele Proiectului

NetworkDevicesMonitor a fost dezvoltat pentru a adresa aceste provocari prin urmatoarele obiective tehnice specifice:

1. Colectare Hibrida și Flexibila:

- Integrarea Syslog (RFC 5424) pe portul 1514 pentru echipamente standard.
- Implementarea de Agenti custom pe portul 9000 pentru metrici detaliate (CPU, RAM, disk).

- Control total prin protocolul JUNK pe portul 8080.
- 2. **Procesare Concurenta Performanta:**
 - Utilizarea I/O Multiplexing cu `poll()` pentru receptie non-blocanta a datelor.
 - Implementarea Thread-Safe Queues pentru decuplarea totala a ingestiei de procesare.
 - Folosirea de Worker threads pentru parsing si analiza paralela.
 - Sincronizare prin Shared mutex pentru operatiuni concurrent read-heavy.
- 3. **Securitate prin Filtrare Multi-Nivel:**
 - Source Manager dedicat cu Whitelist/Blacklist IP-uri.
 - Validare automata a surselor inainte de alocarea resurselor de procesare.
 - Izolare trafic malitios si unknown sources.
 - Filtre personalizate pe tip, mesaj si alerte custom.
- 4. **Vizualizare si UX Avansat:**
 - Dashboard asincron Qt6 cu actualizari in timp real.
 - Grafice interactive (Donut Charts, Line Charts) și sistem de alertare pop-up.

2 Tehnologii Aplicate

Selectia tehnologiilor a fost motivata de nevoia de performanta, control asupra memoriei si interfata moderna.

2.1 C++23 (Backend) și Concurenta

Am ales C++23 pentru a beneficia de noile standarde de siguranta și viteza. Arhitectura se bazeaza pe:

- **Multi-threading:** Am folosit pattern-ul **Producer-Consumer** pentru a gestiona fluxurile de date. Firele de executie sunt gestionate manual pentru un control fin.
- **Sincronizare:** Accesul la structurile de date partajate (liste de loguri, cozi) este protejat prin `std::mutex` și `std::shared_mutex`, asigurand integritatea datelor fara a bloca excesiv citirile.

2.2 Protocolul TCP

Am optat exclusiv pentru TCP (Transmission Control Protocol) pentru toate cele 3 canale (Syslog, Agenti, Admin) din urmatoarele motive:

- Fiabilitate: Este critic ca log-urile de securitate și alertele sa ajunga la destinatie fara pierderi.
- Ordinea pachetelor: Cronologia evenimentelor este esentiala in debugging.

2.3 SQLite3 (Stocare)

Baza de date functioneaza in modul WAL (Write-Ahead Logging). Aceasta este o componenta critica pentru performanta intr-un mediu multithreaded, permitand operatiuni de citire și scriere simultane. Tabelele principale includ: `logs`, `agents_metrics`, `whitelist`, `blacklist`, `alerts` și tabele pentru filtre.

2.4 Qt6 (Interfata Grafica)

Framework-ul Qt6 a fost ales pentru sistemul sau de Signals & Slots, care permite actualizarea interfeței grafice asincron, fara a bloca thread-ul principal (UI thread) atunci cand sosesc pachete de la server.

3 Structura Aplicatiei

Arhitectura este modulara, separand clar logica de backend de interfata de utilizare.

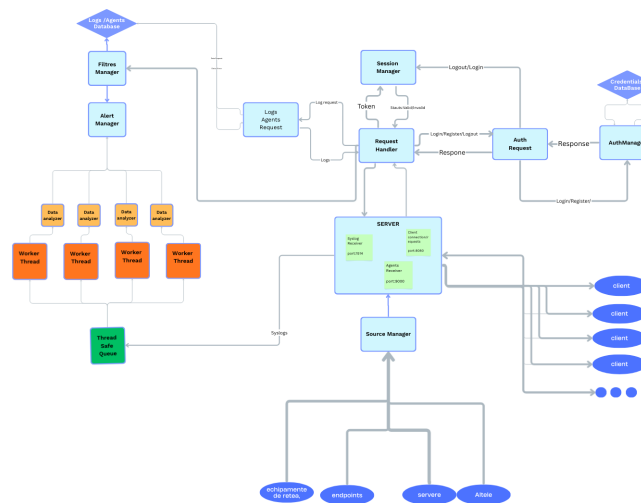


Fig. 1: Architettura completa NetworkDevicesMonitor

3.1 Server Backend

Responsabil pentru logica de business și persistenta datelor.

- **LogPipeline:** Orchestreaza intregul flux de date.
- **SyslogReceiver & AgentsReceiver:** Module de receptie care pun datele in cozi thread-safe.

- **SourceManager:** Gestioneaza securitatea (Whitelist/Blacklist).
- **DataProcessor:** Preia datele din cozi, le parseaza și le trimite la baza de date.
- **AlertsManager:** Analizeaza datele procesate pentru a detecta pattern-uri suspecte (consum mare de resurse, cuvinte cheie interzise).
- **FiltresManager:** Aplica regulile de filtrare definite de utilizator (pe tip sau mesaj).

3.2 Client GUI

O aplicatie desktop structurata pe pagini, cu lifecycle management (hooks de `on_enter` și `on_exit`). **Pagini implementate:**

- **ConnectPage & LoginPage:** Gestionarea conexiunii și autentificarii.
- **HomePage & DashboardPage:** Vizualizare date (Syslog Dashboard, Agents Dashboard, Unknown Data).
- **SecurityPage:** Management vizual pentru Whitelist și Blacklist.
- **AlertsPage & FiltresPage:** Configurare alerte și filtre.

4 Protocolul de Comunicare

Sistemul utilizeaza trei protocoale distincte, adaptate fiecarui tip de trafic.

4.1 A. Port 1514 - Syslog (RFC 5424)

Fluxul de procesare:

1. **Receptie:** Thread cu `poll()` monitorizeaza socket-ul.
2. **Validare:** IP-ul este verificat in Whitelist/Blacklist.
3. **Parsare:** Se extrag campurile standard (PRI, Timestamp, Hostname, App-Name, Message).
4. **Filtrare & Analiza:** Se aplica filtrele și se verifica de alerte.

4.2 B. Port 9000 - Agenti

Protocol binar/JSON optimizat:

- Header (4 bytes): Lungimea payload-ului.
- Payload: JSON cu metrice (`cpu`, `ram`, `disk`).

4.3 C. Port 8080 - Admin API (Protocol JUNK)

Protocolul custom JUNK (Just Useful Notation Kinda) este folosit pentru comunicarea Client-Server. Format: Length(4 bytes) + key:{value};key2:{value2};

Tabel complet comenzi disponibile:

Comanda	Format cerere JUNK
Login	type:{login};username:{...};password:{...};
Register	type:{register};username:{...};password:{...};
Get Logs	type:{logs};last_id:{123};token:{...};
Get Agents	type:{agents};last_id:{0};token:{...};
Add Whitelist	type:{add_whitelist};ip:{...};hostname:{...};token:{...};
Add Blacklist	type:{add_blacklist};ip:{...};token:{...};
Remove Whitelist	type:{remove_whitelist};ip:{...};token:{...};
Remove Blacklist	type:{remove_blacklist};ip:{...};token:{...};
Get Alerts	type:{update_alerts};last_id:{0};token:{...};
Resolve Alert	type:{remove_alert};alert_id:{123};token:{...};
Add Type Filter	type:{add_type_filter};keyword:{SSH};token:{...};
Add Msg Filter	type:{add_msg_filter};keyword:{error};token:{...};
Add Custom Alert	type:{add_custom_alert};keyword:{critical};token:{...};
Get Filtres	type:{get_filtres};token:{...};
Unknown Syslog	type:{unknown_syslog};last_id:{0};token:{...};
Unknown Agents	type:{unknown_agents};last_id:{0};token:{...};

Table 1: Specificatia completa a comenzilor API

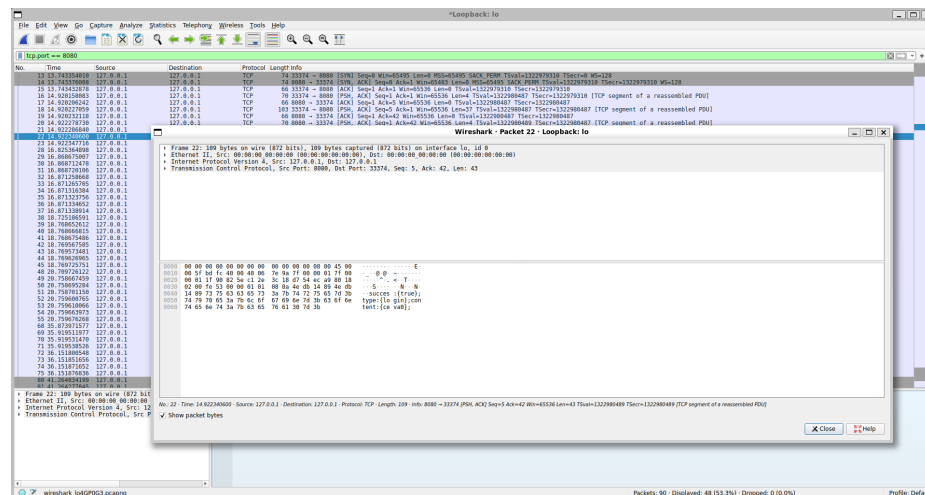


Fig. 2: Captura Wireshark - Analiza pachetelor JUNK

5 Scenarii de Utilizare

5.1 Scenarii de Succes

Scenariu 1: Monitorizare Agent in Timp Real Context: Un server de baze de date PostgreSQL trimite metrice la interval de 5 secunde.

1. Agent Python colecteaza date de sistem (CPU, RAM, Disk).
2. Pachet JSON trimis pe port 9000 cu header binar.
3. Server primeste, valideaza IP in whitelist via SourceManager.
4. DataProcessor extrage metrice, insert in `agents_metrics`.
5. Clientul primeste signal `AgentDataUpdated`.
6. Dashboard actualizeaza graficul CPU/RAM automat fara refresh manual.

Rezultat: Administratorul vizualizeaza in sub 3 secunde cresterea CPU la 92%.

Scenariu 2: Onboarding Echipament Nou Flow:

1. Admin acceseaza SecurityPage → Whitelist tab.
2. Click "Add Source", completeaza in popup IP-ul si Hostname-ul.
3. Click "Add" → cerere JUNK trimisa la server.
4. SourceManager insereaza in DB și actualizeaza hash map-ul in-memory.
5. Switch-ul incepe sa trimita syslog pe 1514. IP-ul este verificat → ACCEPTED.
6. Procesat imediat, apare in Syslog Dashboard.

Rezultat: Zero downtime, vizibilitate instantanee.

Scenariu 3: Configurare Filtre Custom (Reducere Zgomot) Context: Admin vrea sa ignore log-uri verbose de la servere web (HTTP 200 OK).

1. Acceseaza FiltresPage → Message Filters tab.
2. Click "Add Message Filter" și introduce keyword: "HTTP 200".
3. Server adauga in `filtres_message` table și reincarca cache-ul.
4. Log-urile urmatoare cu "HTTP 200" sunt filtrate automat.

Rezultat: Dashboard curat, focus doar pe erori.

Scenariu 4: Alerta Personalizata pe Keyword Context: Admin vrea notificare instant la orice mesaj continand "OutOfMemory".

1. FiltresPage → Custom Alerts → Add "OutOfMemory".
2. Un serviciu Java crasheaza cu "java.lang.OutOfMemoryError".
3. Log syslog primit, DataProcessor face match pe keyword.
4. AlertsManager creeaza alerta CRITICAL.
5. Popup instant la client: "OutOfMemory error detected on app-server-03".

5.2 Scenarii de Eșec (Recuperare și Stabilitate)

Scenariu 5: Trafic Malitios Blocat Context: Botnet incearca flood de date false pe port 1514.

1. 1000 de pachete/sec de pe IP 198.51.100.10 (care este in blacklist).
2. SyslogReceiver detecteaza conexiune in `accept()`.
3. SourceManager verifica: `check_ip_blacklist() → true`.
4. Socket inchis imediat cu `close(client_fd)`, zero procesare.
5. Log intern in `security_events`.

Rezultat: Zero impact pe performanta, CPU usage normal.

Scenariu 6: Pachet Corupt Primit Context: Eroare de retea corupe header-ul unui pachet JUNK.

1. Client trimite pachet cu lungime incorecta in header (0xFFFFFFFF).
2. Server citește 4 bytes: `length = 4294967295` (invalid).
3. Buffer allocation failsafe: maxim 64KB hardcoded.
4. Alocare refuzata, citire partiala/anulata.
5. RequestHandler logheaza eroarea și trimite raspuns de eroare.

Rezultat: Server nu crasheaza (segmentation fault evitat), conexiune mentinuta.

Scenariu 7: Token Expirat Context: Admin lasa aplicatia deschisa 24h, sesiune expirata pe server.

1. Client incearca sa ceara logs cu token vechi.
2. SessionManager verifica: token nu mai exista in hash map (șters automat dupa 12h).
3. Raspuns: eroare, sesiune expirata.

Rezultat: Security enforcement, clientul este delogat automat.

6 Concluzii

Acest proiect a fost, inainte de toate, o experienta de invatare foarte utila. Daca la inceput totul era doar o schita, acum am un server functional (cu Syslog si Agenti) si un client care chiar comunica intre ei.

6.1 Analiza Realizarii

Nu a fost doar despre scris cod, ci despre a intelege cum se leaga lucrurile in prezent:

- **C++23:** Am profitat de ocazie sa explorez noutatile din noul standard pentru a scrie codul mai modern.

- **Concurenta:** Am invatat sa gestionez mai bine firele de executie și sa evit race conditions folosind mutex-uri partajate.
- **Arhitectura:** Am pus mare pret pe modularitate. Faptul ca am impartit totul pe clase si module (SourceManager, AlertsManager, etc.) m-a ajutat enorm sa nu ma pierd in detalii pe masura ce proiectul crestea.

6.2 imbunatatiri Viitoare

Sunt constient ca proiectul nu este perfect. Daca as mai avea timp, urmatoarele aspecte ar putea fi optimizate:

1. **Securitate Transport:** Implementarea SSL/TLS pentru criptarea pachetelor JUNK, care momentan circula in plain-text.
2. **Export Rapoarte:** Adaugarea posibilitatii de a exporta datele din tabele in format CSV sau PDF.

Totusi, pentru stadiul actual, consider ca a iesit un proiect bun, stabil si, cel mai important, usor de inteles datorita structurii sale ordonate. A fost o provocare care mi-a aratat exact unde mai am de lucrat, dar si cat de mult am progresat.

References

1. Facultatea de Informatica Iasi: Retele de Calculatoare - Resurse Curs si Laborator. <https://edu.info.uaic.ro/computer-networks/cursullaboratorul.php>
2. GeeksforGeeks: Multithreading in C++. <https://www.geeksforgeeks.org/cpp/multithreading-in-cpp/>
3. C++ Reference: std::expected (Error handling). <https://en.cppreference.com/w/cpp/utility/expected.html>
4. Splunk: What is Syslog? A Complete Guide to Syslog Monitoring. https://www.splunk.com/en_us/blog/learn/syslog.html
5. Qt Documentation <https://doc.qt.io/>
6. Json Documentation <https://json.nlohmann.me/>
7. Sqlite <https://sqlite.org/cli.html>